

SIL3-STM32-WSW

Dokumentacja projektu systemu komunikacji Master-Slave z mechanizmami bezpieczeństwa

Projekt przedstawia demonstrator systemu dwuprocesorowego, w którym układ Slave wykonuje operacje wyjściowe dopiero po uzyskaniu autoryzacji od układu Master. System wykorzystuje komunikację UART, mechanizm heartbeat, watchdog komunikacyjny oraz CRC dla wybranych ramek transmisji.

Skład grupy

Imię i nazwisko	Numer indeksu	Funkcja w projekcie
Mateusz Skipor	102292	Scrum master
Grzegorz Rak	104706	Product owner
Tobiasz Kandziora	105390	Programista 1 - główny programista, poprawy i wdrożenie kodu programisty 2 oraz program Master
Jakub Krawczyk	104648	Programista 2 - program Slave

Spis treści

1 Zakres dokumentacji	2
2 Wykorzystane zasoby projektu	2
2.1 Pliki opisowe i graficzne	2
2.2 Pliki kodu źródłowego	2
3 Projekt oprogramowania w formie blokowej	2
3.1 Opis architektury	2
3.2 Blokowy projekt systemu	3
3.3 Schematy blokowe działania programów	4
3.4 Pinout układów Master i Slave	6
4 Maszyny stanów	6
4.1 Maszyna stanów układu Slave	6
4.2 Maszyna stanów układu Master	6
4.3 Maszyna stanów układu Slave	7
4.4 Uproszczona maszyna stanów całego urządzenia	8
5 Projekt komunikacji międzyprocesorowej	9
5.1 Kanały komunikacyjne	9
5.2 Format heartbeat	9
5.3 Format żądania autoryzacji	10
5.4 Format odpowiedzi autoryzacyjnej	10
5.5 Uzasadnienie obecnej formy CRC i protokołu	10
6 Fragmenty kodu dokumentujące komunikację	11
6.1 Wysłanie żądania autoryzacji przez Slave	11
6.2 Walidacja żądania autoryzacji przez Master	11
6.3 Heartbeat Mastera	12
6.4 Weryfikacja heartbeat po stronie Slave	12
6.5 Wykonanie operacji LED po autoryzacji	13
7 Analiza zagrożeń	14
8 Analiza bezpieczeństwa projektu	14
9 Wymagania szczegółowe i dowody spełnienia	15
10 Dowody fotograficzne	16
10.1 Zachowanie terminala	16
10.2 Działanie LED	17
10.3 Przerwanie komunikacji UART	17
11 Ograniczenia obecnej wersji projektu	17
12 Wnioski	18

1 Zakres dokumentacji

Dokumentacja obejmuje analizę faktycznego stanu projektu SIL3-STM32-WSW. Uwzględniono projekt oprogramowania w formie blokowej, projekt komunikacji międzyprocesorowej, analizę zagrożeń, analizę bezpieczeństwa oraz wymagania szczegółowe wraz z dowodami ich spełnienia. Dowodami są fragmenty kodu źródłowego oraz zdjęcia znajdujące się w repozytorium projektu.

2 Wykorzystane zasoby projektu

W dokumentacji wykorzystano wybrane pliki projektowe, kod źródłowy oraz zdjęcia dokumentujące działanie systemu. Pominęto pliki środowiska STM32 HAL, pliki historii, pliki wynikowe kompilacji oraz elementy, które nie zawierają właściwej logiki projektu.

2.1 Pliki opisowe i graficzne

- README.md - główny opis projektu, założenia architektury, opis komunikacji i testów.
- docs/System Hardware Diagram.png - diagram sprzętowy systemu.
- photos/MX_Pinout_Master.png - zrzut konfiguracji pinów układu Master.
- photos/MX_Pinout_Slave.png - zrzut konfiguracji pinów układu Slave.
- photos/Komendy z konsoli (z komentarzami).png - dowód zachowania systemu widoczny w terminalu.
- photos/Komendy z konsoli (bez komentarzy).png - dowód zachowania systemu bez dodatkowych opisów.
- photos/Led1.png oraz photos/Led2.png - dowody fotograficzne działania wyjść LED.
- photos/PrzerwanyUART.png - dowód reakcji na przerwanie komunikacji UART.
- photos/Schemat blokowy działania Master.png - schemat blokowy działania programu Master.
- photos/Schemat blokowy działania Slave.png - schemat blokowy działania programu Slave.

2.2 Pliki kodu źródłowego

- master_fw/UART_MASTER/Core/Src/master_app.c - główna logika aplikacji Master.
- master_fw/UART_MASTER/Core/Src/sil_master.c - moduł heartbeat i CRC po stronie Master.
- slave_fw/UART_SLAVE/Core/Src/slave_app.c - główna maszyna stanów, obsługa komend i autoryzacji po stronie Slave.
- slave_fw/UART_SLAVE/Core/Src/sil_watchdog.c - watchdog komunikacyjny i weryfikacja ramek heartbeat po stronie Slave.
- slave_fw/UART_SLAVE/Core/Src/led_driver.c - sterowanie diodami LED i self-test wyjść.

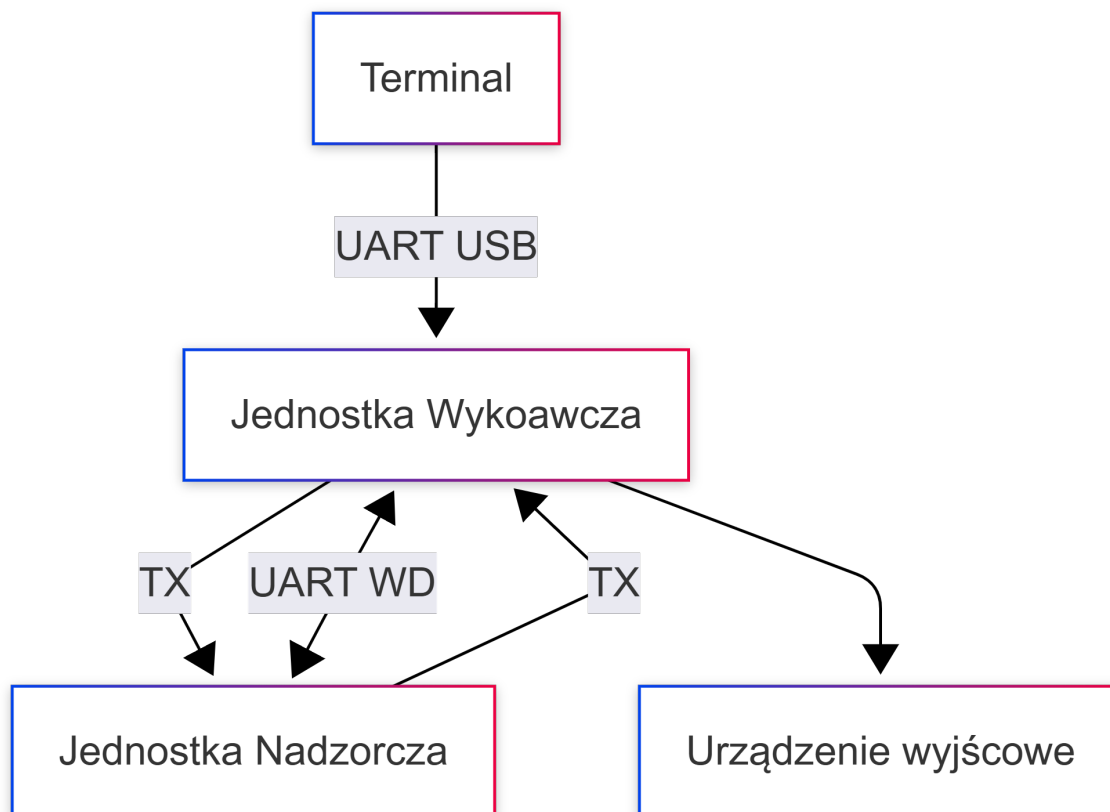
3 Projekt oprogramowania w formie blokowej

3.1 Opis architektury

System jest podzielony na dwa główne mikrokontrolery. Pierwszym jest Master, czyli jednostka nadzorująca odpowiedzialna za wysyłanie heartbeat i autoryzowanie operacji. Drugim jest Slave, czyli jednostka wykonawcza odbierająca polecenia od użytkownika i sterująca LED-ami.

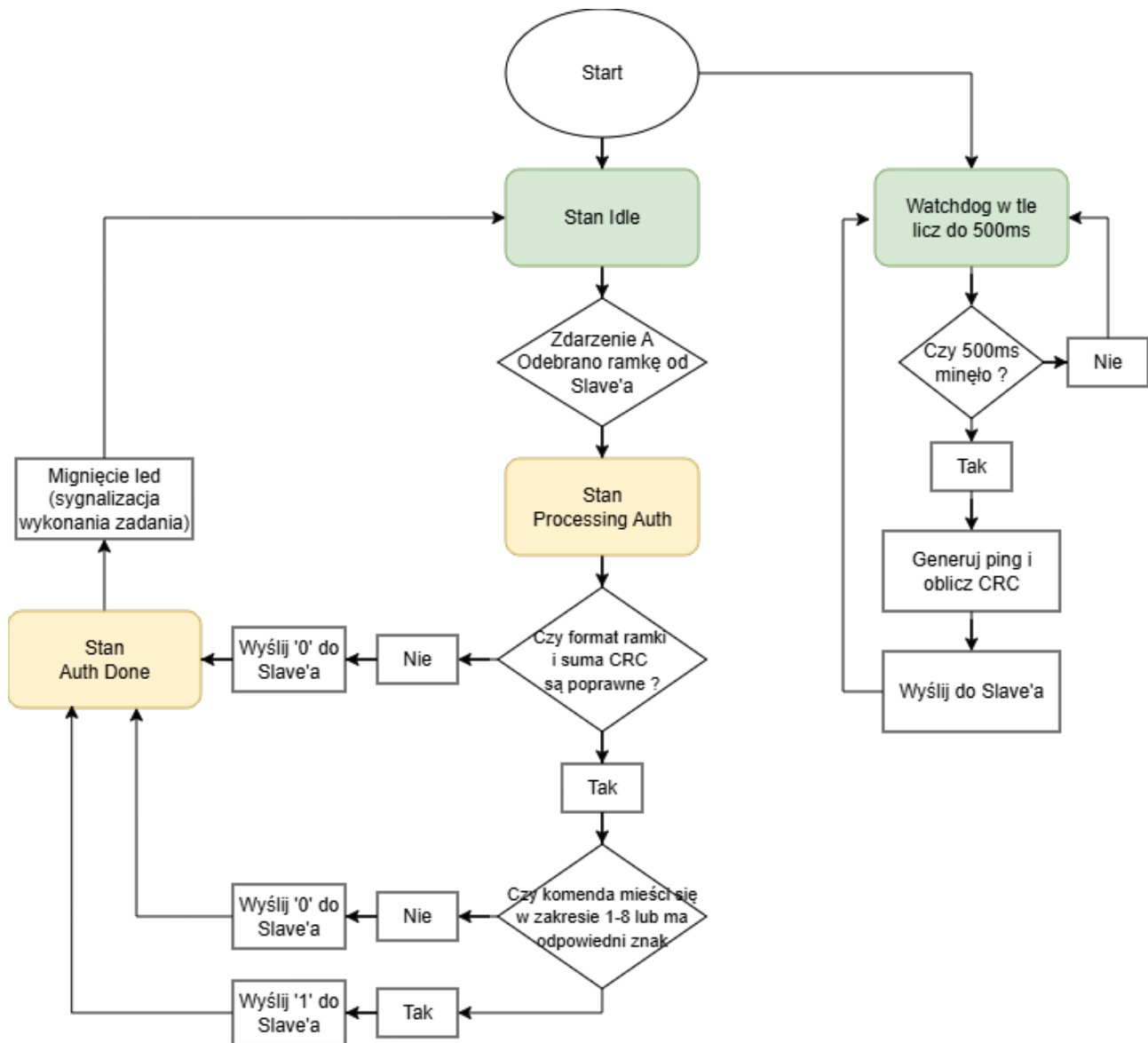
Operator komunikuje się z układem Slave przez UART. Slave nie wykonuje komendy bezpośrednio, tylko wysyła do Mastera zadanie autoryzacji. Dopiero po odebraniu pozytywnej odpowiedzi od Mastera wykonywana jest operacja na wyjściu LED.

3.2 Blokowy projekt systemu

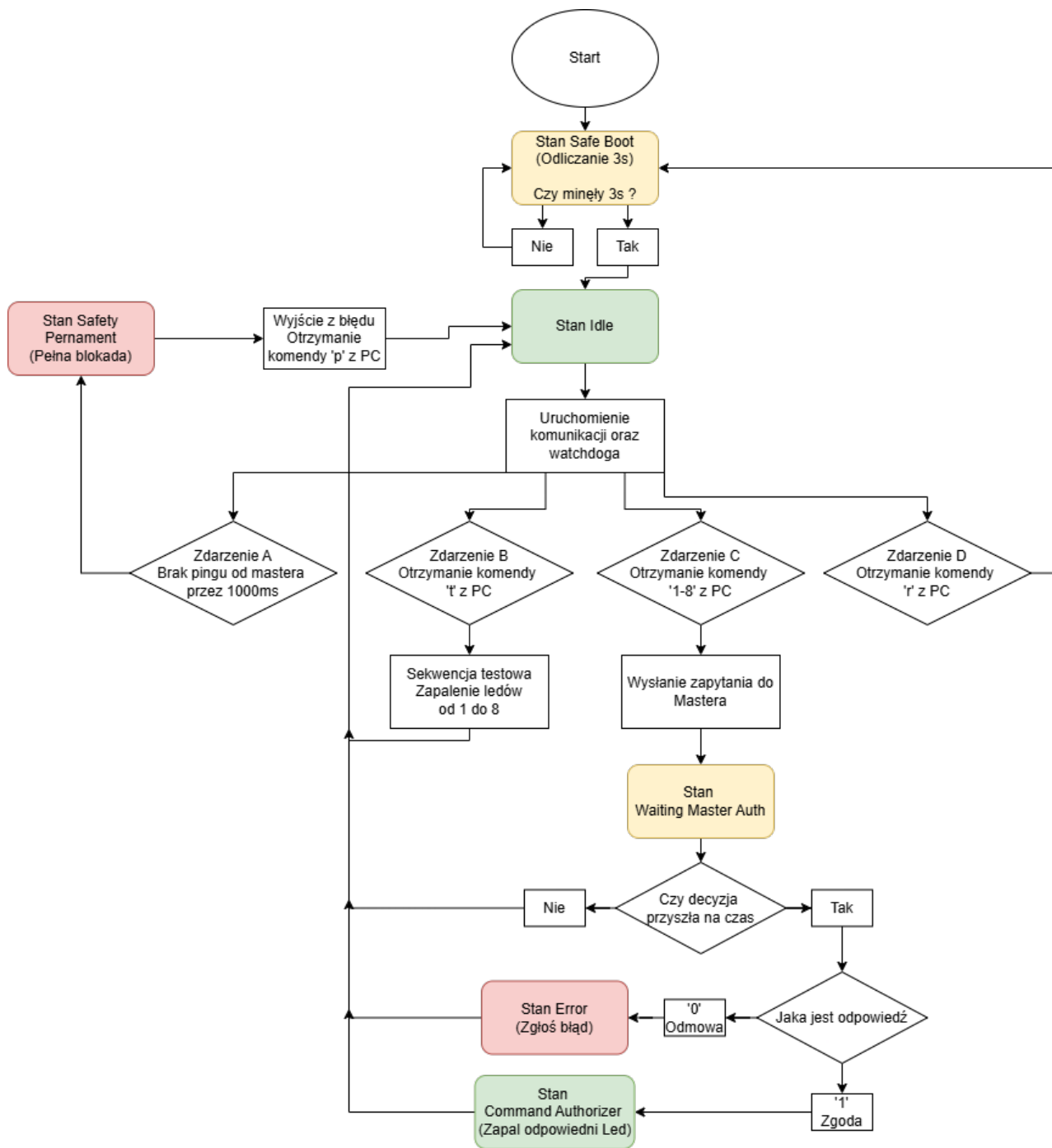


Rys. 1: Diagram sprzętowy systemu

3.3 Schematy blokowe działania programów

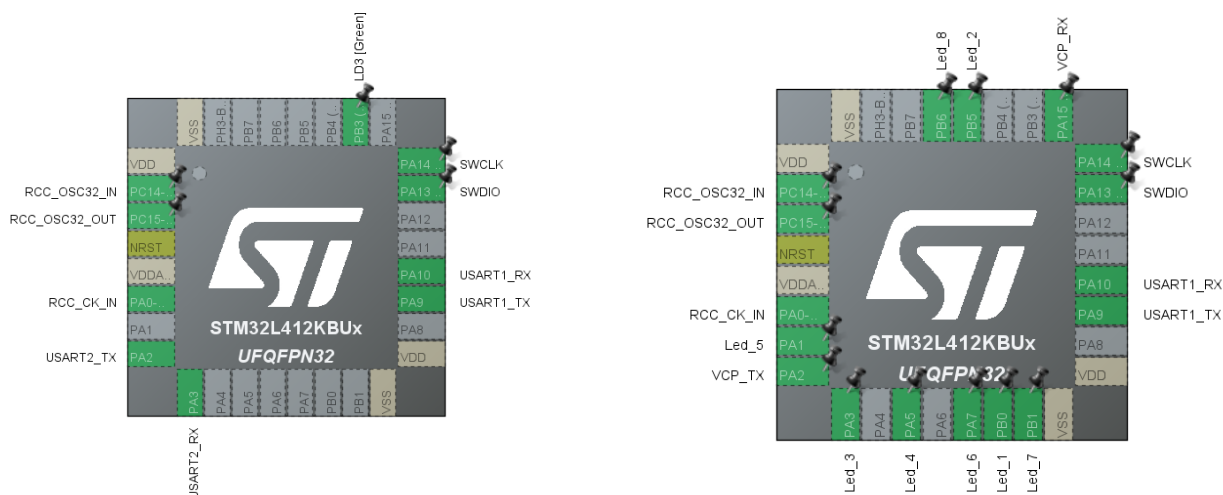


Rys. 2: Schemat blokowy działania programu Master



Rys. 3: Schemat blokowy działania programu Slave

3.4 Pinout układów Master i Slave



Rys. 4: Konfiguracja pinów układów Master i Slave. Po lewej stronie przedstawiono konfigurację układu Master, a po prawej konfigurację układu Slave.

4 Maszyny stanów

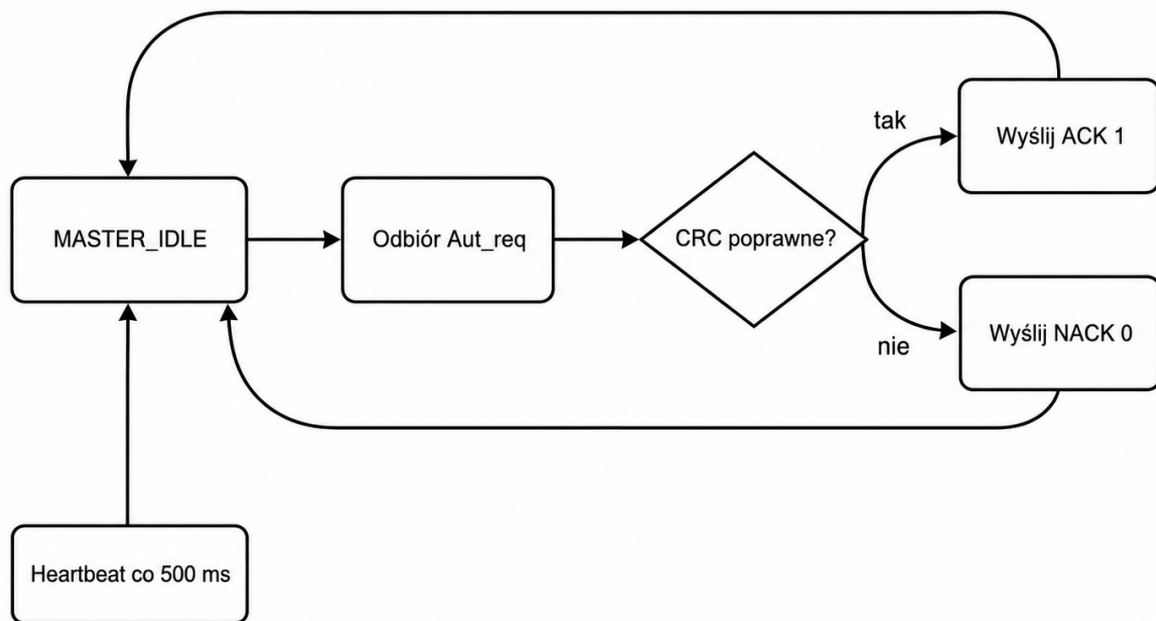
4.1 Maszyna stanów układu Slave

Slave pracuje jako jednostka wykonawcza. Najważniejsze stany programu Slave to STATE_SAFETY, STATE_IDLE, STATE_WAITING_MASTER_AUTH, STATE_COMMAND_AUTHORIZED, STATE_ERROR oraz STATE_SAFETY_PERMANENT.

Stan STATE_SAFETY odpowiada za bezpieczny start po uruchomieniu. Stan STATE_IDLE oznacza gotowość do przyjęcia komendy. Stan STATE_WAITING_MASTER_AUTH oznacza oczekiwanie na odpowiedź Mastera. Stan STATE_COMMAND_AUTHORIZED oznacza, że operacja została zaakceptowana. Stan STATE_ERROR oznacza błąd autoryzacji. Stan STATE_SAFETY_PERMANENT oznacza trwały stan bezpieczny po utracie heartbeat.

4.2 Maszyna stanów układu Master

Master odbiera zadania autoryzacyjne, sprawdza format oraz CRC, a następnie odsyła odpowiedź pozytywną albo negatywną. Dodatkowo generuje cykliczny heartbeat wysyłany do układu Slave.

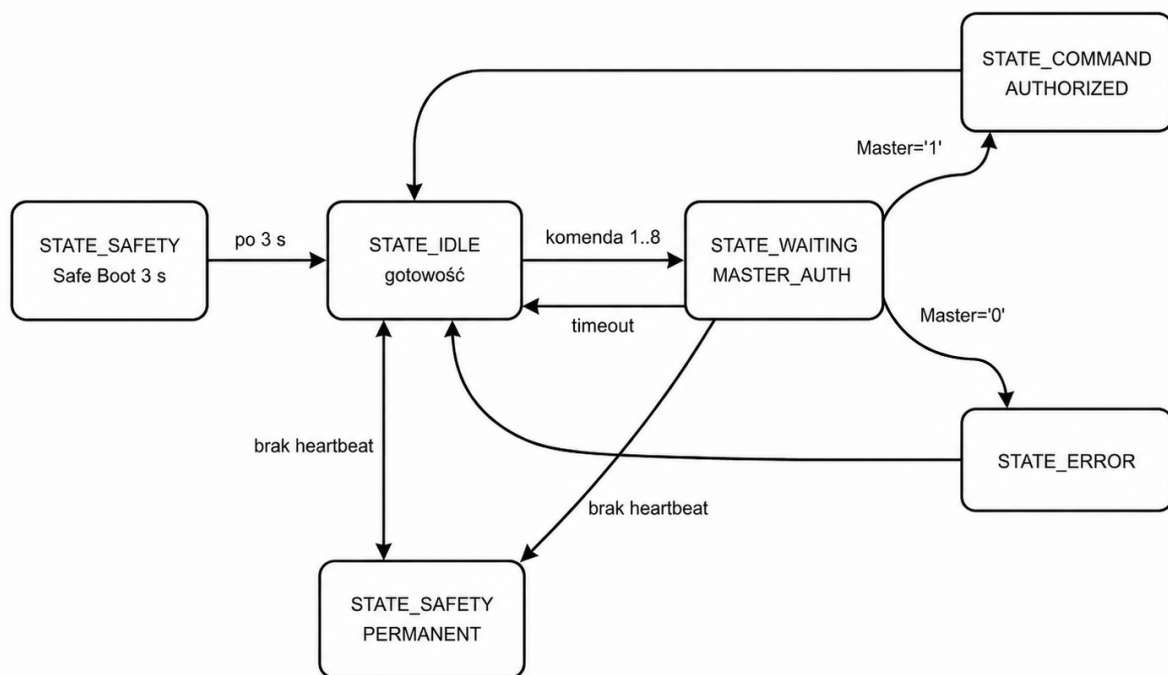


Uproszczona maszyna stanów układu Master

Rys. 5: Uproszczona maszyna stanów układu Master.

4.3 Maszyna stanów układu Slave

Slave pracuje jako jednostka wykonawcza. Po uruchomieniu przechodzi przez stan bezpiecznego startu, a następnie oczekuje na komendy użytkownika. Po odebraniu komendy Slave wysyła do Mastera żądanie autoryzacji i czeka na odpowiedź. W przypadku pozytywnej odpowiedzi wykonuje operację, natomiast w przypadku odmowy lub błędu przechodzi do obsługi stanu błędu. Utrata heartbeat powoduje przejście do trwałego stanu bezpiecznego.

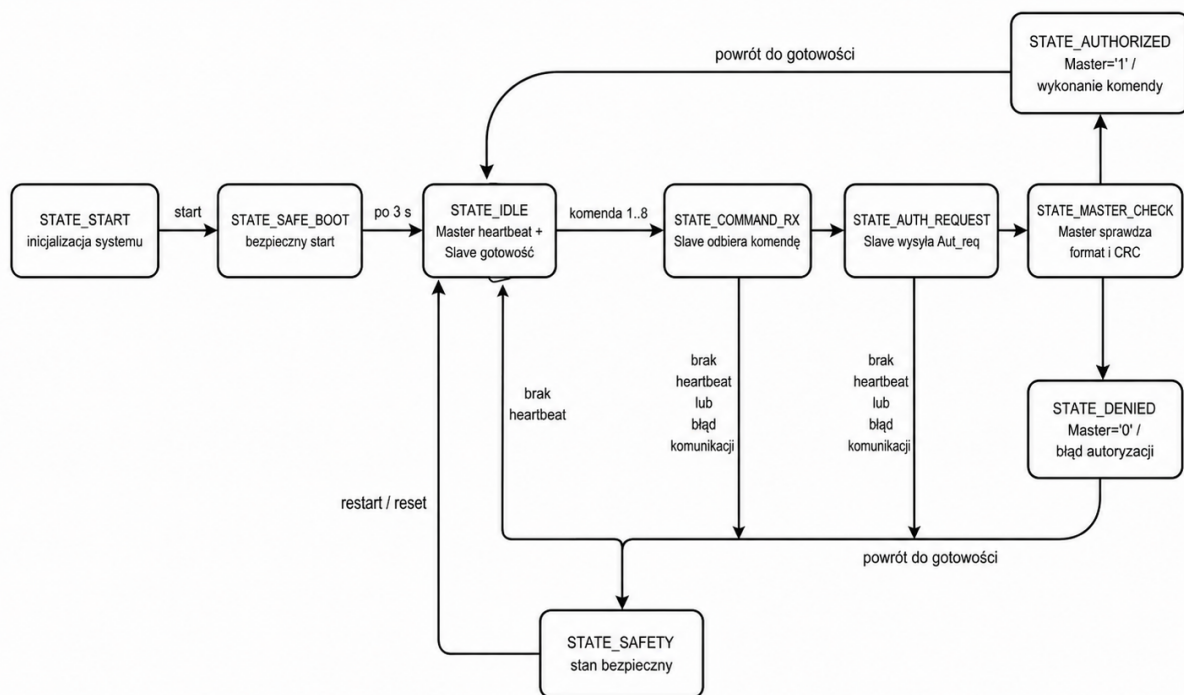


Uproszczona maszyna stanów układu Slave

Rys. 6: Uproszczona maszyna stanów układu Slave.

4.4 Uproszczona maszyna stanów całego urządzenia

Całe urządzenie można opisać jako połączenie działania układu Master oraz układu Slave. Po starcie system przechodzi przez bezpieczną inicjalizację, następnie znajduje się w stanie gotowości, w którym Master wysyła heartbeat, a Slave oczekuje na polecenia użytkownika. Po odebraniu komendy Slave wysyła żądanie autoryzacji, Master sprawdza poprawność ramki oraz CRC, a następnie zwraca decyzję. W przypadku braku heartbeat lub błędu komunikacji system przechodzi do stanu bezpiecznego.



Uproszczona maszyna stanów całego urządzenia Master-Slave

Rys. 7: Uproszczona maszyna stanów całego urządzenia Master-Slave.

5 Projekt komunikacji międzyprocesorowej

5.1 Kanały komunikacyjne

Tabela 1: Interfejsy komunikacyjne systemu

Interfejs	Nadawca	Odbiorca	Funkcja
UART2 Slave	PC	Slave	Komendy operatora, komunikaty tekstowe.
UART1 Slave-Master	Slave	Master	Zadania autoryzacji.
UART1 Master-Slave	Master	Slave	Heartbeat oraz odpowiedzi autoryzacyjne.
UART2 Master	Master	Terminal diagnostyczny	Komunikaty debug i diagnostyka.

Parametry UART użyte w projekcie to 115200 bit/s, 8 bitów danych, brak parzystości, 1 bit stopu oraz brak sprzętowej kontroli przepływu.

5.2 Format heartbeat

Ramka heartbeat wysyłana jest przez Mastera do Slave co 500 ms. Ma długość 4 bajtów.

Tabela 2: Format ramki heartbeat

Bajt	Znaczenie	Opis
0	'H'	Znak startowy heartbeat.
1	sequence	Numer sekwencyjny.
2	crc high	Starszy bajt CRC.
3	crc low	Młodszy bajt CRC.

5.3 Format żądania autoryzacji

Slave wysyła do Mastera żądania w postaci tekstowej.

```
Aut_req:<numer_led>,<crc>\r\n
```

Kod 1: Format żądania autoryzacji

CRC jest liczone z jednego bajtu komendy. Master po odebraniu ramki sprawdza prefiks, numer komendy oraz zgodność CRC.

5.4 Format odpowiedzi autoryzacyjnej

Master odsyła odpowiedź tekstową. Odpowiedź 1 oznacza zgodę na wykonanie operacji, natomiast odpowiedź 0 oznacza odmowę wykonania operacji.

Odpowiedź autoryzacyjna w obecnej wersji projektu nie zawiera własnego CRC, numeru sekwencyjnego ani identyfikatora zadania. Jest to faktyczny stan projektu i należy go traktować jako ograniczenie obecnej implementacji.

5.5 Uzasadnienie obecnej formy CRC i protokołu

Mechanizm CRC został zrealizowany w obecnej, uproszczonej formie ze względu na ograniczony czas realizacji projektu. Bardziej rozbudowana wersja CRC i protokołu komunikacyjnego była przygotowywana, jednak zespół programistów nie był w stanie w wyznaczonym czasie uruchomić poprawnie działającej, rozbudowanej wersji. Z tego powodu w finalnym stanie projektu pozostawiono rozwiązanie prostsze, ale działające i możliwe do udokumentowania.

Dodatkowym czynnikiem wpływającym na zakres dopracowania kodu były problemy organizacyjne i techniczne wynikające z nieznamości środowiska zarządzania projektem Git. Problemy te realnie skróciły czas, który mógł zostać przeznaczony na dopracowanie kodu, testowanie i uporządkowanie wspólnych modułów komunikacyjnych.

6 Fragmenty kodu dokumentujące komunikację

6.1 Wysłanie żądania autoryzacji przez Slave

```
1 static void SendAuthorizationRequest(uint8_t command)
2 {
3     uint16_t crc=SIL_CalculateCRC(&command,1);
4     char buffer[24];
5     uint8_t len=snprintf(buffer, sizeof(buffer), "Aut_req:%d,%04X\r\n", command, crc);
6
7     HAL_UART_Transmit(&huart1, (uint8_t*)buffer, len, 100);
8
9     slave_state=STATE_WAITING_MASTER_AUTH;
10    auth_timeout=HAL_GetTick();
11 }
```

Kod 2: Slave - wysłanie żądania autoryzacji do Mastera

6.2 Walidacja żądania autoryzacji przez Master

```
1 if (strncmp((char*)rx_buffer, "Aut_req:", 8)==0)
2 {
3     int command=0;
4     uint16_t received_crc=0;
5     char *ptr=(char*)&rx_buffer[8];
6
7     command=atoi(ptr);
8
9     while (*ptr && *ptr != ',')
10    {
11        ptr++;
12    }
13
14    if (*ptr == ',')
15    {
16        ptr++;
17    }
18
19    received_crc=(uint16_t)strtol(ptr, NULL, 16);
20
21    if (command > 0 && command <= 8)
22    {
23        uint8_t cmd_byte=(uint8_t)command;
24        uint16_t calculated_crc=SIL_CalculateCRC(&cmd_byte, 1);
25
26        if (calculated_crc == received_crc)
27        {
28            ProcessAuthorizationRequest(command);
29        }
30        else
31        {
32            SendAuthorizationResponse(0);
33        }
34    }
35 }
```

```

34     }
35 }

```

Kod 3: Master - sprawdzenie żądania autoryzacji

6.3 Heartbeat Mastera

```

1  #define PING_INTERVAL_MS 500
2
3  void SIL_Master_Process(void)
4  {
5      if ((HAL_GetTick()-last_ping_sent_time) >= PING_INTERVAL_MS)
6      {
7          uint8_t ping_frame[4];
8
9          ping_frame[0]='H';
10         ping_frame[1]=ping_sequence;
11
12         uint16_t crc=SIL_CalculateCRC(ping_frame,2);
13
14         ping_frame[2]=(uint8_t)((crc>>8)&0xFF);
15         ping_frame[3]=(uint8_t)(crc&0xFF);
16
17         HAL_UART_Transmit(&huart1,ping_frame,4,100);
18
19         ping_sequence++;
20         last_ping_sent_time=HAL_GetTick();
21     }
22 }

```

Kod 4: Master - generowanie heartbeat

6.4 Weryfikacja heartbeat po stronie Slave

```

1  SIL_Status_t SIL_ProcessPingFrame(uint8_t* frame,uint8_t length)
2  {
3      if (length != 4)
4      {
5          return SIL_ERROR;
6      }
7
8      if (frame[0] != 'H')
9      {
10         return SIL_ERROR;
11     }
12
13     uint16_t received_crc=(frame[2]<<8)|frame[3];
14     uint16_t calculated_crc=SIL_CalculateCRC(frame,2);
15
16     if (calculated_crc != received_crc)
17     {
18         return SIL_ERROR;
19     }
20 }

```

```

19     }
20
21     if (frame[1] == last_sequence)
22     {
23         return SIL_ERROR;
24     }
25
26     last_sequence=frame[1];
27     SIL_Watchdog_Feed();
28
29     return SIL_OK;
30 }

```

Kod 5: Slave - sprawdzenie ramki heartbeat

6.5 Wykonanie operacji LED po autoryzacji

```

1 void SlaveApp_UART1_RxCallback(uint8_t rx_byte)
2 {
3     if (ping_rx_index == 0)
4     {
5         if (rx_byte == 'H')
6         {
7             ping_rx_buffer[0]=rx_byte;
8             ping_rx_index=1;
9         }
10        else if (rx_byte == '0' || rx_byte == '1')
11        {
12            if (slave_state == STATE_WAITING_MASTER_AUTH)
13            {
14                if (rx_byte == '1')
15                {
16                    LED_Set(leds_to_set);
17                    SendMessage("Operacja wykonana!\r\n");
18                    slave_state=STATE_COMMAND_AUTHORIZED;
19                }
20                else
21                {
22                    SendMessage("BLAD: Brak autoryzacji!\r\n");
23                    slave_state=STATE_ERROR;
24                }
25            }
26        }
27    }
28 }

```

Kod 6: Slave - wykonanie operacji po odpowiedzi Mastera

7 Analiza zagrożeń

Tabela 3: Analiza zagrożeń projektu

Zagrożenie	Opis	Obecny mechanizm
Podszybie się pod Mastera	Atakujący może próbować udawać Mastera i wysłać do Slave bajt 1.	Slave sprawdza tylko, czy znajduje się w stanie oczekiwania na autoryzację.
Manipulacja danymi	Możliwa jest zmiana ramki Aut_req lub heartbeat podczas transmisji.	CRC wykrywa błędy przypadkowe w wybranych ramach.
Powtórzenie ramki	Możliwe jest powtórzenie starej odpowiedzi autoryzacyjnej.	Heartbeat ma numer sekwencyjny, ale odpowiedź autoryzacyjna go nie posiada.
Utrata komunikacji	Możliwe jest przerwanie przewodu UART lub zatrzymanie Mastera.	Slave wykrywa brak heartbeat i przechodzi do stanu bezpiecznego.
Brak trwałego audytu	Po restarcie nie zostaje trwały ślad przyczyny awarii.	Komunikaty są wysyłane tylko przez UART.

8 Analiza bezpieczeństwa projektu

Projekt zawiera kilka istotnych mechanizmów ograniczających ryzyko. Slave nie wykonuje komendy LED bez zapytania Mastera o zgodę. Master wysyła cykliczny heartbeat. Slave wykrywa brak heartbeat i przechodzi do stanu bezpiecznego. Ramka heartbeat zawiera CRC, a zadanie autoryzacji również zawiera CRC. Po utracie komunikacji Slave gasi wyjścia LED. System posiada także self-test LED.

Jednocześnie obecna wersja projektu ma ograniczenia wynikające ze stanu implementacji. Odpowiedź Mastera 0/1 nie zawiera CRC, numeru sekwencyjnego ani identyfikatora zadania. CRC zabezpiecza wybrane ramki przed błędami transmisji, ale nie stanowi mechanizmu kryptograficznego. Mechanizm timeoutu autoryzacji po stronie Mastera jest niepełny, ponieważ stan MASTER_PROCESSING_AUTH nie jest realnie wykorzystywany w głównym przepływie działania programu.

Ograniczenia te wynikają z faktycznego przebiegu realizacji projektu. Bardziej rozbudowana wersja mechanizmu CRC i komunikacji była planowana, jednak nie została poprawnie uruchomiona w wyznaczonym czasie. Dodatkowo część czasu projektowego została utracona na problemy związane z obsługą Git i zarządzaniem repozytorium, co ograniczyło czas dostępny na dopracowanie kodu oraz dokumentacji.

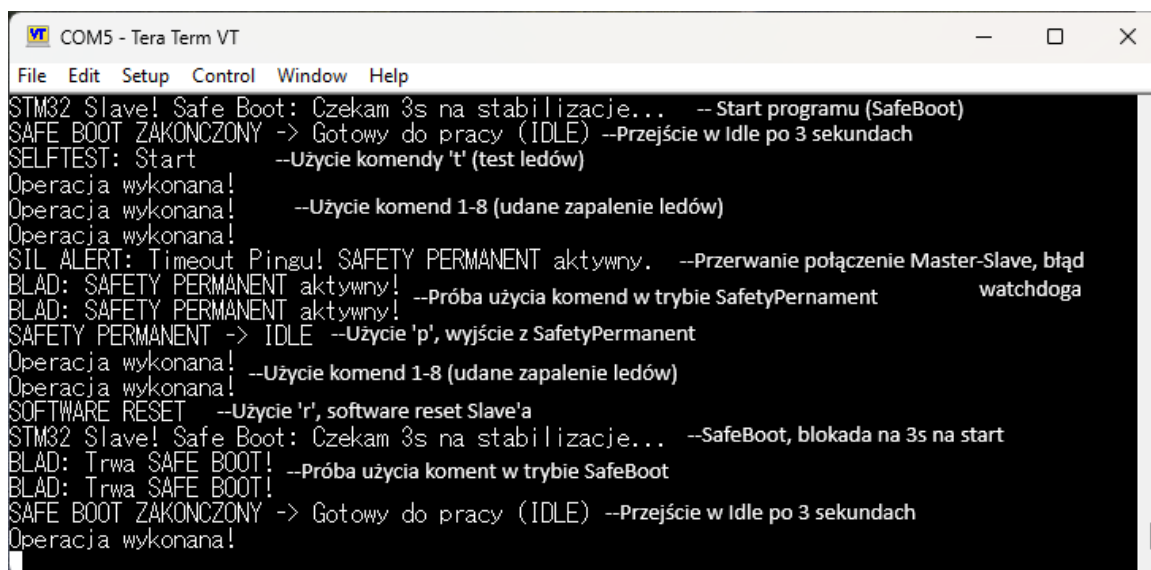
9 Wymagania szczegółowe i dowody spełnienia

Tabela 4: Wymagania szczegółowe projektu

ID	Wymaganie	Status	Dowód
REQ-01	Slave po starcie przechodzi do trybu bezpiecznego i czeka 3 s na stabilizację.	Spełnione	slave_app.c, funkcje SlaveApp_Init i SlaveApp_Process.
REQ-02	Slave nie wykonuje komendy LED bez wysłania zadania autoryzacji do Mastera.	Spełnione	SendAuthorizationRequest.
REQ-03	Zadanie autoryzacji zawiera CRC.	Spełnione	Obliczanie CRC w SendAuthorizationRequest.
REQ-04	Master sprawdza format Aut_req.	Spełnione	Parser w master_app.c.
REQ-05	Master sprawdza CRC zadania autoryzacji.	Spełnione	Porównanie calculated_crc i received_crc.
REQ-06	Master wysyła heartbeat co 500 ms.	Spełnione	SIL_Master_Process.
REQ-07	Heartbeat zawiera numer sekwencyjny.	Spełnione	Zmienna ping_sequence.
REQ-08	Heartbeat zawiera CRC.	Spełnione	SIL_CalculateCRC.
REQ-09	Slave sprawdza CRC heartbeat.	Spełnione	SIL_ProcessPingFrame.
REQ-10	Slave przechodzi do stanu bezpiecznego po utracie heartbeat.	Spełnione	SIL_Watchdog_IsExpired.
REQ-11	Po utracie heartbeat wyjścia LED są gaszone.	Spełnione	Wywołanie LED_Set(0).
REQ-12	Slave wykonuje self-test wyjść LED.	Spełnione	LED_SelfTest.
REQ-13	System dokumentuje zachowanie w terminalu.	Spełnione częściowo	Zdjęcia z konsoli.
REQ-14	System dokumentuje fizyczne działanie LED.	Spełnione częściowo	Zdjęcia Led1.png i Led2.png.
REQ-15	Odpowiedź Mastera jest powiązana z aktualnym stanem oczekiwania Slave na autoryzację.	Spełnione częściowo	Stan Slave ogranicza kontekst, ale protokół nie zawiera ID zadania.
REQ-16	Odpowiedź Mastera jest zabezpieczona CRC.	Niespełnione	Master wysyła tylko 1 albo 0.

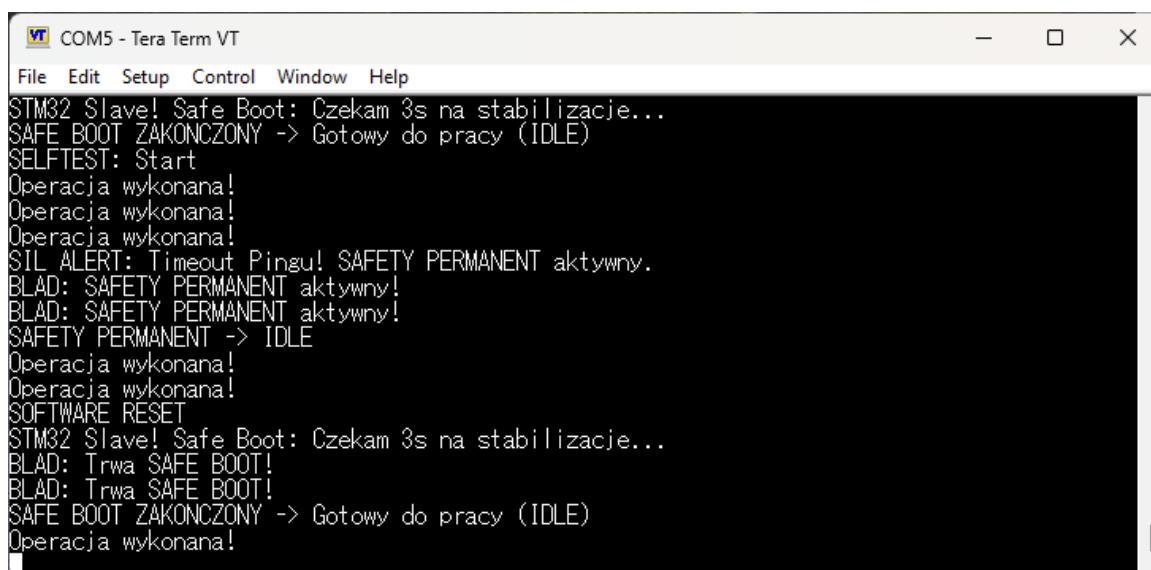
10 Dowody fotograficzne

10.1 Zachowanie terminala



```
COM5 - Tera Term VT
File Edit Setup Control Window Help
STM32 Slave! Safe Boot: Czekam 3s na stabilizacje... --Start programu (SafeBoot)
SAFE BOOT ZAKONCZONY -> Gotowy do pracy (IDLE) --Przejscie w Idle po 3 sekundach
SELFTEST: Start --Uzycie komendy 't' (test ledów)
Operacja wykonana!
Operacja wykonana! --Uzycie komend 1-8 (udane zapalenie ledów)
Operacja wykonana!
SIL ALERT: Timeout Pingu! SAFETY PERMANENT aktywny. --Przerwanie połączenie Master-Slave, błąd
BLAD: SAFETY PERMANENT aktywny! --Próba użycia komend w trybie SafetyPermant watchdoga
BLAD: SAFETY PERMANENT aktywny!
SAFETY PERMANENT -> IDLE --Uzycie 'p', wyjście z SafetyPermanent
Operacja wykonana! --Uzycie komend 1-8 (udane zapalenie ledów)
Operacja wykonana!
SOFTWARE RESET --Uzycie 'r', software reset Slave'a
STM32 Slave! Safe Boot: Czekam 3s na stabilizacje... --SafeBoot, blokada na 3s na start
BLAD: Trwa SAFE BOOT! --Próba użycia koment w trybie SafeBoot
BLAD: Trwa SAFE BOOT!
SAFE BOOT ZAKONCZONY -> Gotowy do pracy (IDLE) --Przejscie w Idle po 3 sekundach
Operacja wykonana!
```

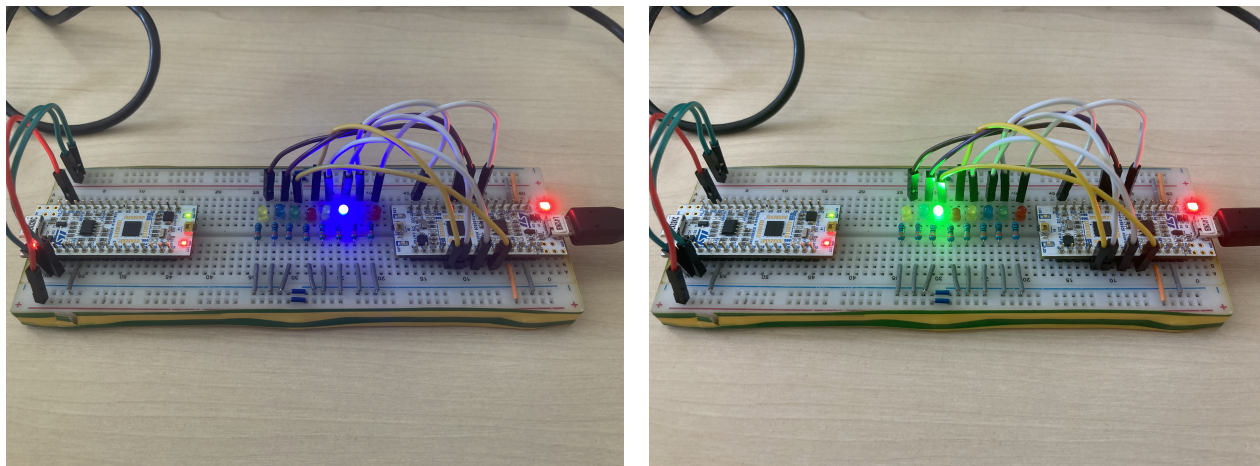
Rys. 8: Zachowanie systemu widoczne w terminalu - wersja z komentarzami



```
COM5 - Tera Term VT
File Edit Setup Control Window Help
STM32 Slave! Safe Boot: Czekam 3s na stabilizacje...
SAFE BOOT ZAKONCZONY -> Gotowy do pracy (IDLE)
SELFTEST: Start
Operacja wykonana!
Operacja wykonana!
Operacja wykonana!
SIL ALERT: Timeout Pingu! SAFETY PERMANENT aktywny.
BLAD: SAFETY PERMANENT aktywny!
BLAD: SAFETY PERMANENT aktywny!
SAFETY PERMANENT -> IDLE
Operacja wykonana!
Operacja wykonana!
SOFTWARE RESET
STM32 Slave! Safe Boot: Czekam 3s na stabilizacje...
BLAD: Trwa SAFE BOOT!
BLAD: Trwa SAFE BOOT!
SAFE BOOT ZAKONCZONY -> Gotowy do pracy (IDLE)
Operacja wykonana!
```

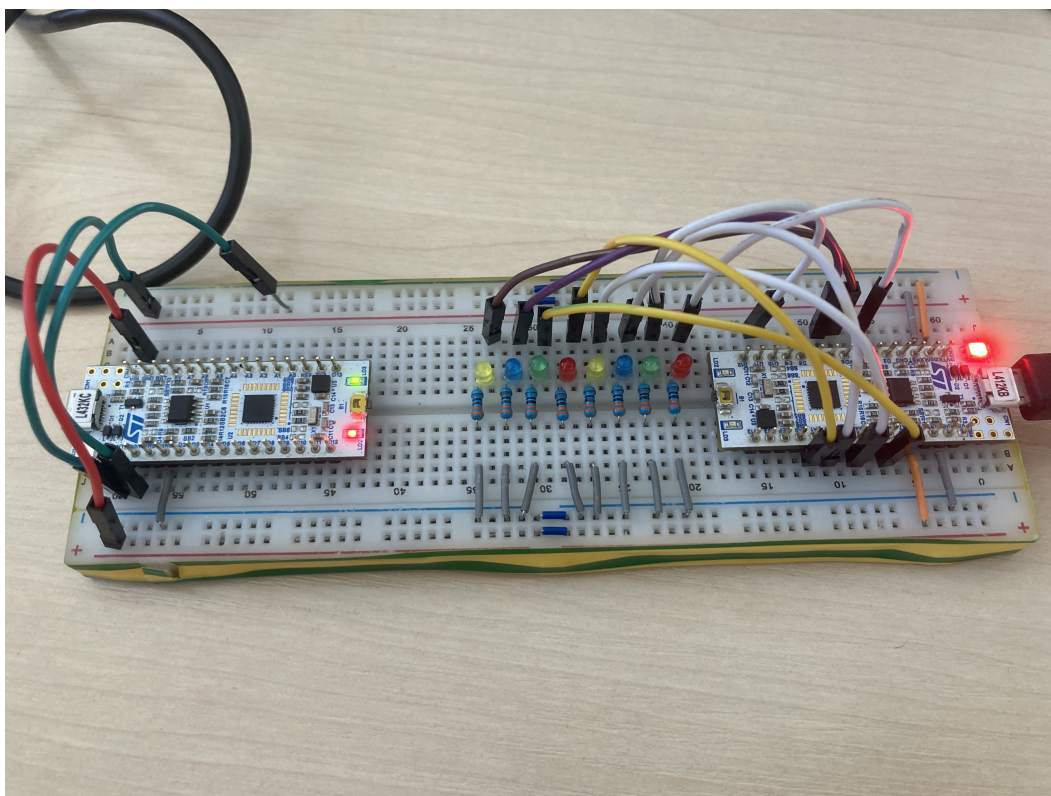
Rys. 9: Zachowanie systemu widoczne w terminalu - wersja bez komentarzy

10.2 Działanie LED



Rys. 10: Dowody fotograficzne działania wyjść LED. Zdjęcia przedstawiają przykładowe stany wyjść po wykonaniu komendy przez układ Slave.

10.3 Przerwanie komunikacji UART



Rys. 11: Dowód reakcji systemu na przerwanie komunikacji UART

11 Ograniczenia obecnej wersji projektu

Obecna wersja projektu dobrze przedstawia ideę komunikacji nadzorowanej i reakcję na utratę heartbeat. Nie stanowi jednak kompletnego, certyfikowanego rozwiązania SIL3. W projekcie brakuje pełnej formalnej analizy niezawodności, pełnej dokumentacji testów, kryptograficznego uwierzytelnienia komunikacji oraz trwałego logowania zdarzeń.

Najważniejsze ograniczenie techniczne dotyczy protokołu autoryzacji. Slave wykonuje operację po odebraniu bajtu 1, jeżeli znajduje się w stanie oczekiwania na autoryzację. Mechanizm ten pokazuje zasadę działania systemu Master-Slave, ale nie zapewnia pełnej identyfikacji odpowiedzi z konkretnym żądaniem.

Kolejne ograniczenie dotyczy CRC. CRC zostało zastosowane dla heartbeat oraz zadania autoryzacji, jednak nie obejmuje odpowiedzi Mastera. Wynika to z faktu, że bardziej rozbudowana wersja nie została poprawnie uruchomiona w czasie przewidzianym na realizację projektu. Decyzja o pozostawieniu prostszej wersji pozwoliła zakończyć projekt w stanie działającym i możliwym do zaprezentowania.

12 Wnioski

Projekt realizuje podstawową ideę bezpiecznego nadzoru: jednostka wykonawcza nie wykonuje operacji samodzielnie, lecz wymaga zgody jednostki nadrzędnej. Dodatkowo zastosowano heartbeat, watchdog komunikacyjny, CRC oraz reakcję bezpieczną po utracie komunikacji. Mechanizmy te tworzą działający demonstrator systemu nadzorowanego.

Obecny stan projektu należy jednak traktować jako demonstrację zasady działania, a nie jako pełny system SIL3. Najważniejsze ograniczenia wynikają z uproszczonego protokołu autoryzacji, niepełnego objęcia odpowiedzi Mastera przez CRC oraz problemów czasowych i organizacyjnych podczas realizacji projektu. Mimo tych ograniczeń projekt spełnia podstawowe założenia demonstracyjne i posiada materiał dowodowy w postaci kodu oraz zdjęć dokumentujących zachowanie systemu.